

LAPORAN PERANCANGAN

Smart Product Digital: CornGuard Sistem Deteksi Dini Multi-Penyakit pada Tanaman Jagung Berbasis Computer Vision

Disusun Oleh :

Aan Krisnawati	NPM: 24782001
Dinda Tiara Putri	NPM: 24782008
Juwita Valentiya	NPM: 24782014
Nabhilla Ayu Larasati	NPM: 24782022



Dosen Mata Kuliah:

Ir. Nurul Qomariyah, S.Kom., M.Kom

Dian Ayu Afifah, S.Si., M.Sc.

**PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027**

DAFTAR ISI

1.	Ringkasan Eksekutif (Executive Summary)	1
2.	Arsitektur Sistem (System Architecture)	2
2.1	Diagram Arsitektur Sistem & Alur Kerja	2
2.1.1	Pemisahan Dataset dan Training Model	3
2.2	Spesifikasi Teknologi (Tech Stack)	5
2.2.1	Perangkat Keras/Sensor	5
2.2.2	Backend dan API	5
2.2.3	Kecerdasan Buatan/Algoritma/Model/Dataset.....	5
2.2.4	Frontend/Antarmuka	5
2.2.5	Komponen Utama Aplikasi.....	6
3.	Rancangan Fitur Inti (Core Features).....	6
3.1	Deteksi Dini Multi-Penyakit (Hybrid Classification-Detection)	6
3.2	Rekomendasi Penanganan Adaptif	6
3.3	Saran Perawatan Preventif	7
3.4	Pembelajaran Berkelanjutan dari Data Historis	7
4.	Alur Data dan Cara Kerja (System Workflow).....	8
4.1	Pengumpulan Data (Data Collection)	8
4.2	Transmisi dan Komunikasi (Transmission)	8
4.3	Pemrosesan (Processing).....	9
4.4	Eksekusi dan Tampilan (Action/Display)	9
5.	Pengujian dan Limitasi (Testing and Limitation)	9
5.1	Metrik Pengujian.....	9
5.2	Limitasi Saat Ini	10

SMART DIGITAL PRODUCT: CornGuard Sistem Deteksi Dini Multi-Penyakit pada Tanaman Jagung Berbasis Computer Vision

1. Ringkasan Eksekutif (Executive Summary)

1. Masalah

Mengidentifikasi secara dini penyakit pada tanaman jagung. Untuk membantu para petani jagung, mahasiswa/akademisi pertanian dan perkebunan, serta pelaku usaha tanaman jagung agar bisa lebih awal menyadari adanya penyakit pada tanaman jagung sehingga tidak mengalami kerugian yang jauh lebih besar dan bisa ditangani segera.

2. Solusi Digital

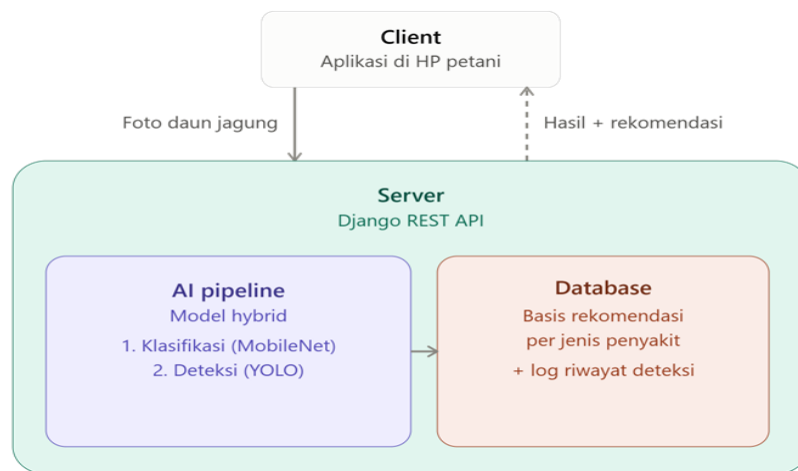
Di sini kami memberikan sebuah solusi, yaitu CornGuard, sistem berbasis AI yang digunakan untuk mendeteksi secara dini penyakit pada tanaman jagung dan mengelompokkannya ke dalam beberapa kategori penyakit jagung seperti Blight, Bulai, Common Rust, Gray Leaf Spot, dan Healthy (Sehat). Lalu, sistem CornGuard juga bisa memberikan rekomendasi solusi untuk menangani masalah-masalah pada kategori penyakit tanaman jagung.

3. Target Pengguna

1. Petani Jagung
2. Mahasiswa/Akademisi Pertanian-Perkebunan
3. Pelaku Usaha Tani Jagung

2. Arsitektur Sistem (System Architecture)

2.1 Diagram Arsitektur Sistem & Alur Kerja



Gambar 2.1 Arsitektur dan Alur Kerja Sistem Deteksi Dini Multi-Penyakit Tanaman Jagung

Berdasarkan Gambar 2.1 yang memberikan gambaran keterkaitan antara Client dan Server pada Sistem Deteksi Dini Multi-penyakit pada Tanaman Jagung (CornGuard), berikut penjelasan peran dan hubungan masing-masing komponen.

Client pada sistem CornGuard berperan sebagai antarmuka (UI) yang digunakan langsung oleh pengguna (petani jagung, mahasiswa pertanian/Perkebunan, atau pelaku usaha tani jagung). Melalui Client, pengguna dapat mengunggah foto daun jagung yang ingin didiagnosis, kemudian menerima hasil diagnosis beserta rekomendasi penanganan untuk tanaman jagung yang terdeteksi penyakit dengan penyampaian solusi dalam bentuk yang mudah dipahami bagi semua kalangan. Client tidak melakukan pemrosesan data maupun pengambilan Keputusan, seluruh proses dilakukan sepenuhnya oleh Server, sejalan dengan pendekatan *centralized architecture* (Sebuah sistem di mana satu server pusat yang mengontrol seluruh pemrosesan data, penyimpanan, dan pengelolaan aturan di dalam jaringan) yang diterapkan pada sistem CornGuard.

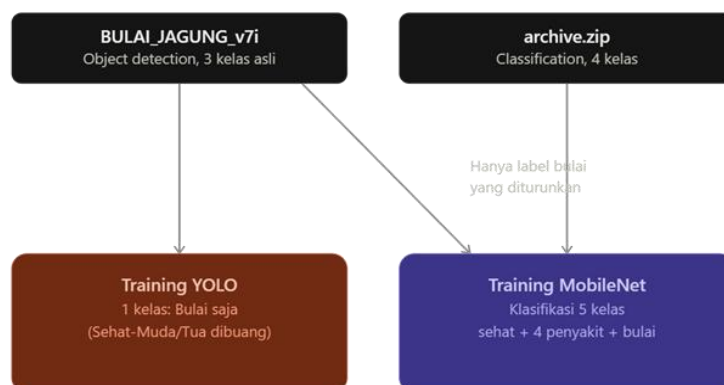
Server berperan sebagai pusat pemrosesan seluruh data dan logika sistem CornGuard, dibangun menggunakan Django REST API sebagai penghubung antara Client dan komponen-komponen di dalam server. Jadi, ketika server menerima input sebuah foto daun jagung dari Client, Django REST API akan meneruskan foto tersebut ke AI pipeline (serangkaian langkah terstruktur yang saling terhubung) untuk dianalisis.

Di dalam Server, AI Pipeline bekerja menggunakan pendekatan hybrid yang terdiri atas dua tahap. Tahap pertama menggunakan model klasifikasi (MobileNet) untuk melakukan identifikasi awal secara cepat, yaitu menentukan apakah tanaman dalam kondisi sehat atau menunjukkan gejala salah satu penyakit. Apabila hasil klasifikasi menunjukkan adanya indikasi penyakit, foto akan diteruskan ke tahap kedua, yaitu model deteksi (YOLO), yang bertugas menandai lokasi spesifik gejala penyakit pada daun secara lebih presisi (Istilah 'presisi' merujuk ke pada ketepatan lokasi spasial (koordinat box hasil YOLO)).

Setelah AI Pipeline menghasilkan keputusan jenis penyakit, Django REST API akan mengambil informasi rekomendasi penanganan yang sesuai dari Database. Database pada sistem ini menyimpan basis pengetahuan rekomendasi penanganan untuk setiap jenis penyakit, serta menyimpan log riwayat hasil deteksi yang nantinya dapat dimanfaatkan sebagai data tambahan untuk pelatihan ulang model di masa mendatang.

Seluruh hasil dari proses tersebut, jenis penyakit, lokasi gejala, dan rekomendasi penanganan yang kemudian dirangkum oleh Django REST API dan dikirimkan kembali ke Client untuk ditampilkan kepada pengguna.

2.1.1 Pemisahan Dataset dan Training Model



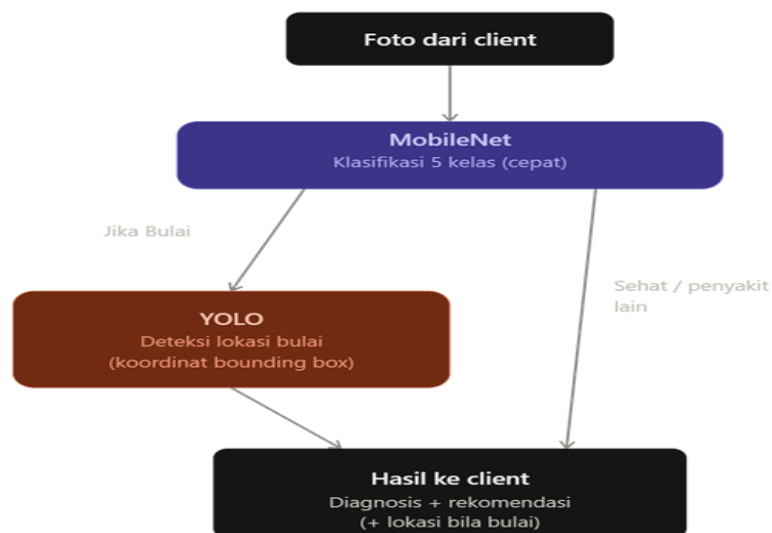
Gambar 2.2 Alur Pemisahan Dataset dan Pelatihan Model (Training Phase)

Pada Gambar 2.2, sistem CornGuard melatih dua model secara terpisah karena keduanya berasal dari dua tipe dataset dengan format anotasi yang berbeda dan tidak bisa disatukan dalam satu proses pelatihan.

1. **Pelatihan Model Klasifikasi (MobileNet).** Dilatih dari gabungan data satu-label-per-gambar: empat kelas dari archive.zip (Blight, Common Rust, Gray Leaf Spot,

Healthy), ditambah label Bulai yang diturunkan otomatis dari dataset object detection BULAI_JAGUNG_v7i (lihat Gambar 2.2), setiap foto dengan minimal satu bounding box "Bulai" dikonversi menjadi satu label "Bulai". Hasilnya, MobileNet menjadi classifier 5 kelas: Healthy, Blight, Common Rust, Gray Leaf Spot, dan Bulai.

2. **Pelatihan Model Deteksi Objek (YOLO).** Hanya dilatih dari dataset yang memiliki anotasi bounding box, yaitu BULAI_JAGUNG_v7i. Dari 3 kelas bounding box aslinya (Bulai, Sehat-Muda, Sehat-Tua), dua kelas sehat sengaja dikeluarkan, sehingga YOLO dilatih sebagai detector 1 kelas saja: Bulai. Karena dalam alur kerja sistem, YOLO memang tidak pernah dipanggil untuk kasus daun sehat.



Gambar 2.3 Alur Kerja saat Sistem Berjalan

Mekanisme kerja saat sistem berjalan yang ditunjukkan pada Gambar 2.3, begitu kedua model selesai dilatih, keduanya dijalankan secara berurutan di server setiap ada foto masuk:

1. Foto diproses lebih dulu oleh MobileNet, yang langsung menentukan salah satu dari 5 kelas.
2. Jika hasilnya Bulai, foto diteruskan ke YOLO untuk menandai koordinat lokasi gejala secara spesifik pada daun.

3. Jika hasilnya Healthy atau salah satu dari 3 penyakit lain, proses berhenti di situ. Hasil klasifikasi beserta rekomendasi langsung dikirim ke Client, tanpa localization (karena YOLO memang tidak dilatih untuk 3 penyakit lainnya (Blight, Common Rust, dan Gray Leaf Spot), hal ini dicatat sebagai keterbatasan sistem CornGuard, bukan disembunyikan).

2.2 Spesifikasi Teknologi (Tech Stack)

2.2.1 Perangkat Keras/Sensor

Perangkat keras yang digunakan berupa smartphone/tablet milik pengguna, tanpa sensor tambahan khusus. Kamera bawaan berperan menangkap citra daun jagung sebagai data masukan utama, sementara layar menampilkan antarmuka aplikasi beserta hasil diagnosis.

2.2.2 Backend dan API

Django REST Framework, berperan sebagai pusat pemrosesan seluruh logika sistem dan penghubung antara Client, AI Pipeline, serta Database.

2.2.3 Kecerdasan Buatan/Algoritma/Model/Dataset

Pendekatan hybrid dua tahap, yaitu MobileNet untuk klasifikasi cepat (5 kelas: Healthy, Blight, Common Rust, Gray Leaf Spot, Bulai) dan YOLOv8 untuk deteksi lokasi gejala (1 kelas: Bulai). Dataset merupakan gabungan archive.zip (classification, 4 kelas) dan BULAI_JAGUNG_v7i (object detection), dengan proses pemisahan data pelatihan sebagaimana dijelaskan pada Gambar 2.2 dan 2.3.

2.2.4 Frontend/Antarmuka

Antarmuka CornGuard dibangun sebagai aplikasi mobile berbasis Flutter, sehingga dapat berjalan pada platform Android maupun iOS dari satu basis kode yang sama. Aplikasi berperan sebagai titik interaksi utama antara pengguna dan sistem: mengaktifkan kamera atau galeri untuk mengambil foto daun jagung, mengirimkan foto tersebut ke backend melalui Django REST API dalam format multipart/form-data, lalu menampilkan hasil diagnosis, lokasi gejala (apabila terdeteksi Bulai), dan rekomendasi penanganan dalam tampilan yang sederhana. Aplikasi tidak melakukan pemrosesan maupun validasi citra apa pun di sisi

client, sejalan dengan pendekatan centralized architecture pada CornGuard. Pengguna juga dapat melihat kembali riwayat hasil diagnosis sebelumnya melalui menu riwayat, memanfaatkan data log yang tersimpan di Database backend.

2.2.5 Komponen Utama Aplikasi

(1) modul pengambilan foto, mengakses kamera/galeri perangkat; (2) modul komunikasi, mengirim foto ke endpoint backend dan menangani respons termasuk kondisi gagal koneksi/timeout; (3) modul tampilan hasil, menyajikan jenis penyakit, lokasi gejala, dan rekomendasi; (4) modul riwayat, menampilkan daftar diagnosis yang pernah dilakukan berdasarkan data dari backend.

3. Rancangan Fitur Inti (Core Features)

3.1 Deteksi Dini Multi-Penyakit (Hybrid Classification-Detection)

- 1) Fungsi: mendeteksi secara dini apakah daun jagung dalam kondisi sehat atau menunjukkan gejala salah satu dari empat jenis penyakit (Bulai, Blight, Common Rust, Gray Leaf Spot), sekaligus menandai lokasi gejala secara spesifik (bounding box) khusus untuk kasus Bulai. Jadi bukan cuma melabeli sakit atau tidak, tapi juga menunjukkan di mana letak masalahnya.
- 2) Input data: foto daun jagung yang diunggah pengguna melalui aplikasi.
- 3) Output/aksi: label klasifikasi jenis kondisi daun (sehat atau salah satu penyakit); khusus kasus Bulai, sistem menambahkan koordinat lokasi gejala pada foto.

3.2 Rekomendasi Penanganan Adaptif

- 1) Fungsi: memberikan solusi penanganan yang sesuai dengan jenis penyakit yang terdeteksi secara otomatis, tanpa pengguna perlu mencari sendiri atau menunggu kehadiran ahli pertanian, dengan bahasa penyampaian yang bisa diterima semua kalangan.
- 2) Input data: hasil klasifikasi jenis penyakit dari fitur deteksi (fitur 1).
- 3) Output/aksi: teks rekomendasi penanganan (opsi kimiawi dan non-kimiawi) yang diambil otomatis dari basis pengetahuan sistem sesuai jenis penyakit yang terdeteksi.

3.3 Saran Perawatan Preventif

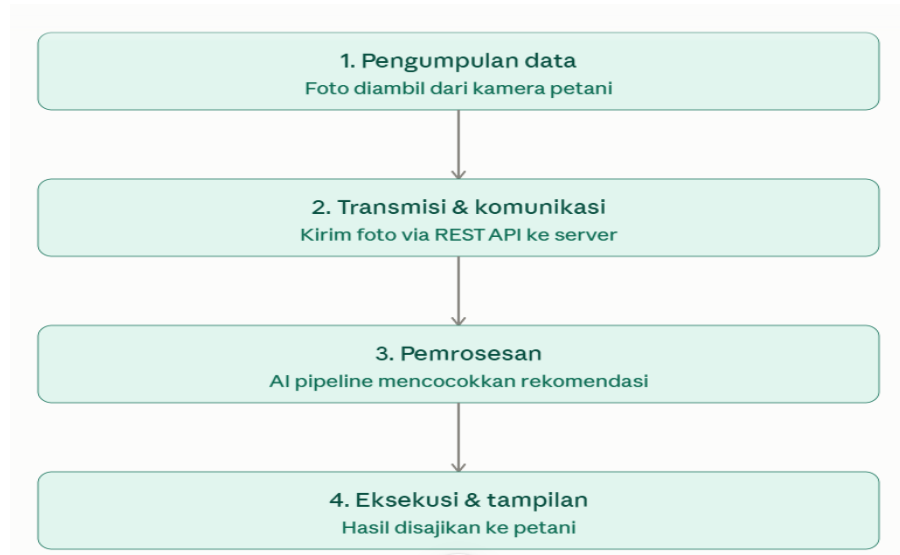
- 1) Fungsi: menjaga tanaman yang sudah terdeteksi sehat agar tetap sehat hingga masa panen, sehingga sistem tidak hanya reaktif terhadap penyakit tapi juga proaktif mencegahnya.
- 2) Input data: hasil klasifikasi "Sehat" dari fitur deteksi (fitur 1).
- 3) Output/aksi: saran perawatan pencegahan (pemupukan, pengairan, jarak tanam, dan sejenisnya) yang ditampilkan kepada pengguna.

3.4 Pembelajaran Berkelanjutan dari Data Historis

- 1) Fungsi: memanfaatkan foto dan hasil deteksi yang terkumpul dari penggunaan sistem sebagai data tambahan untuk melatih ulang model di masa mendatang, sehingga akurasi deteksi bisa terus meningkat seiring bertambahnya data dari kondisi lapangan nyata.
- 2) Input data: log foto beserta hasil klasifikasi/deteksi yang tersimpan di Database setiap kali sistem digunakan.
- 3) Output/aksi: kumpulan data terlabeli yang menjadi kandidat dataset tambahan untuk proses retraining model. *Catatan: fitur ini merupakan rencana pengembangan jangka Panjang/Opsional.*

4. Alur Data dan Cara Kerja (System Workflow)

Secara keseluruhan, cara kerja sistem CornGuard dapat digambarkan sebagai satu rangkaian proses yang berjalan secara berurutan, mulai dari data diperoleh hingga hasil akhirnya disampaikan kepada pengguna. Alur ini terdiri atas empat tahap utama, sebagaimana ditunjukkan pada Gambar 4.1 berikut.



Gambar 4.1 Alur Data dan Cara Kerja (System Workflow)

Gambar 4.1 menunjukkan bahwa keempat tahap berjalan secara berurutan, satu tahap baru dimulai setelah tahap sebelumnya selesai. Foto tanaman jagung dari petani dikumpulkan (pengumpulan data), dikirim ke server melalui REST API (transmisi), dianalisis oleh AI Pipeline hingga menghasilkan diagnosis dan rekomendasi (pemrosesan), lalu hasilnya disajikan kembali kepada petani (eksekusi dan tampilan).

4.1 Pengumpulan Data (Data Collection)

Data mentah pada sistem CornGuard diperoleh langsung dari pengguna, berupa foto tanaman jagung yang diambil menggunakan kamera ponsel petani. Data hanya dikirim ke sistem saat pengguna aktif memotret dan meminta analisis, bukan lewat pembacaan sensor berkala atau pengambilan data otomatis dari sumber lain.

4.2 Transmisi dan Komunikasi (Transmission)

Komunikasi antara aplikasi (client) dan server berjalan melalui arsitektur client-server menggunakan REST API yang dibangun dengan Django REST Framework. Foto yang diambil pengguna dikirim dari aplikasi ke server, dan sebagai balasannya server

mengembalikan hasil analisis dalam format JSON yang mencakup diagnosis beserta rekomendasi penanganan.

4.3 Pemrosesan (Processing)

Di sisi server, foto yang diterima lebih dulu melalui tahap validasi dan preprocessing untuk memastikan kelayakannya sebelum dianalisis. Selanjutnya foto diproses melalui AI Pipeline dengan pendekatan hybrid dua tahap. Tahap pertama menggunakan model klasifikasi (MobileNet) untuk menentukan kondisi kesehatan tanaman secara umum. Apabila hasil klasifikasi mengindikasikan penyakit Bulai, foto diteruskan ke tahap kedua menggunakan model deteksi objek (YOLO) untuk menandai lokasi gejala secara lebih spesifik. Hasil analisis ini kemudian dicocokkan dengan basis data rekomendasi penanganan sesuai jenis penyakit yang terdeteksi.

4.4 Eksekusi dan Tampilan (Action/Display)

Hasil akhir dari proses analisis, yaitu jenis penyakit, lokasi gejala (apabila teridentifikasi), serta rekomendasi penanganan, dikembalikan oleh server ke aplikasi dan ditampilkan kepada pengguna dalam antarmuka yang informatif dan mudah dipahami. Dengan begitu, petani bisa langsung mendapat gambaran kondisi tanamannya beserta langkah tindak lanjut yang perlu dilakukan, tanpa perlu interpretasi teknis lebih lanjut.

5. Pengujian dan Limitasi (Testing and Limitation)

5.1 Metrik Pengujian

Karena proyek masih pada tahap rancangan, metrik berikut merupakan rencana pengujian yang akan dijalankan pada tahap pengembangan teknis, belum hasil aktual.

Tabel 1. Metrik Pengujian Sistem CornGuard

Komponen	Metrik	Apa yang diukur
Model YOLO (deteksi lokasi Bulai)	mAP50	Seberapa sering kotak lokasi gejala yang digambar YOLO tumpang tindih minimal 50% dengan lokasi sebenarnya — ukuran akurasi lokasi yang longgar
	mAP50-95	Rata-rata akurasi lokasi pada berbagai ambang ketepatan (50% sampai 95%) — versi lebih ketat, menilai seberapa presisi kotaknya
Model MobileNet	Akurasi	Persentase foto yang berhasil diklasifikasikan dengan benar dari seluruh foto yang diuji

(klasifikasi 5 kelas)		
	F1-score	Keseimbangan ketepatan dan kelengkapan prediksi per kelas — penting karena jumlah data antar kelas (misalnya Sehat-Muda) tidak seimbang, sehingga akurasi saja bisa menyesatkan
Sistem secara keseluruhan	Response time API	Waktu sejak foto dikirim hingga hasil diterima kembali oleh pengguna, sebagai indikator kelayakan pengalaman pengguna

5.2 Limitasi Saat Ini

Tabel 2. Limitasi Sistem

Limitasi	Penjelasan
Localization terbatas pada Bulai	YOLO hanya dilatih mendeteksi lokasi gejala Bulai. Untuk Blight, Common Rust, dan Gray Leaf Spot, sistem baru bisa memberi hasil klasifikasi tanpa menandai lokasi spesifik, karena ketiganya belum punya data anotasi bounding box
Distribusi kelas belum sepenuhnya seimbang	Sebagian kelas pada dataset (misalnya Sehat-Muda) jumlah datanya lebih sedikit dibanding kelas lain, berpotensi memengaruhi akurasi klasifikasi untuk kelas tersebut
Frontend belum diimplementasikan	Antarmuka aplikasi (Flutter) yang dijelaskan di bagian 2.2.4 masih berupa rancangan, belum berupa aplikasi yang berjalan
Validasi lapangan belum dilakukan	Model belum diuji langsung terhadap foto dari kondisi lapangan sebenarnya, sehingga performa aktual di luar dataset belum bisa dipastikan